

EDGAR Mispricing Pipeline: An Alternative Data System for Prediction Market Signal Detection

RJ Guhl

University of Illinois Urbana-Champaign

Champaign, Illinois, USA

richardjohnghuhl@gmail.com

Abstract

EDGAR Mispricing Pipeline is an end-to-end data pipeline that extracts actionable signals from SEC earnings call transcripts and FRED macroeconomic indicators, then compares calibrated model probabilities against live prediction market contract prices on Kalshi and Polymarket. The system uses the Claude API for structured extraction of sentiment and financial themes from earnings transcripts—a task where traditional NLP underperforms—and feeds these features alongside macro indicators into an XGBoost classifier with Platt scaling for probability calibration. Mispricing signals are flagged when the model's calibrated probability diverges meaningfully from the market price. The goal is to explore whether structured alternative data analysis from financial disclosures can reveal early indicators of market-relevant information before they are reflected in contract premiums.

Keywords: Data Engineering, Prediction Markets, LLM Integration, Probability Calibration, Alternative Data, SEC EDGAR, Feature Engineering

1 Introduction

Prediction markets price discrete real-world events and are increasingly used as ground-truth probability benchmarks in finance and policy research. Unlike traditional financial markets, prediction contracts are often thinly traded and driven by fragmented information from news outlets, social platforms, and financial disclosures. This creates an environment where structured analysis of authoritative text sources may surface edges not yet incorporated into contract prices.

Existing work in quantitative finance has demonstrated the predictive value of earnings call sentiment and analyst report language. However, most approaches treat model output as a standalone signal. EDGAR Mispricing Pipeline extends this by anchoring model output against a live market price, framing the task as mispricing detection rather than raw prediction. This distinction is the core technical and financial contribution of the project.

The system focuses specifically on two high-signal data sources: SEC EDGAR earnings call transcripts and FRED macroeconomic indicators. By combining LLM-based structured extraction from transcripts with traditional feature engineering on macro data, the pipeline produces a calibrated probability model for earnings surprises and surfaces contracts where the model's predicted probability meaningfully diverges from the market price.

2 System Architecture

The system is implemented as a modular Python pipeline with five stages, orchestrated by Apache Airflow and backed by cloud storage and a relational feature store.

2.1 Ingestion Layer

- SEC EDGAR earnings call transcripts fetched for S&P 500 constituents on a rolling 90-day window via the EDGAR REST API

- FRED macroeconomic indicators (CPI, unemployment, yield curve, consumer sentiment) pulled on a scheduled cadence via the FRED API
- Kalshi and Polymarket contract prices retrieved via their respective APIs and stored with timestamps for downstream alignment

2.2 Orchestration

An Apache Airflow DAG runs nightly, triggering ingestion, transformation, scoring, and dashboard refresh in sequence. All raw data is persisted to Amazon S3 with partitioned prefixes by source and date, ensuring reproducibility and enabling backtesting against historical snapshots. A dbt layer transforms raw data into analysis-ready tables in the PostgreSQL feature store, enforcing schema contracts and data quality tests.

2.3 Processing & Feature Engineering

The processing layer operates on two parallel tracks. For earnings transcripts, the Claude API is used for structured extraction: each transcript is processed to produce sentiment scores (bullish/bearish/neutral), key financial theme tags, forward guidance indicators, and management tone classifications. This LLM-based extraction produces features unavailable through traditional NLP methods and is the primary analytical differentiator of the system.

For macroeconomic data, FRED indicators are transformed into rate-of-change features, surprise metrics (actual vs. consensus), and rolling volatility measures. All processed features are written to a PostgreSQL feature store keyed by ticker and event date.

2.4 Modeling

An XGBoost classifier is trained on historical earnings events with features from both transcript extraction and macro indicators. Probability calibration is applied via Platt scaling to ensure model output represents well-calibrated probabilities rather than raw classification scores. A lead-lag analysis module aligns sentiment and theme intensity spikes with subsequent contract price movements to evaluate signal timing.

2.5 Signal & Output

A mispricing signal is flagged when the calibrated model probability diverges from the Kalshi or Polymarket contract price by a configurable threshold. A Streamlit dashboard displays live signals, model vs. market probability, sentiment trends from transcripts, and pipeline health metrics. A backtesting module evaluates historical signal quality using directional accuracy and precision/recall on contract-relevant retrieval.

3 Project Timeline

Phase	Focus	Key Deliverables
1	Data infrastructure (Weeks 1–3)	API setup, Airflow DAG, S3 storage, EDGAR/FRED ingestion, raw data validation
2	Processing & features (Weeks 4–6)	Claude API extraction pipeline, dbt transforms, macro feature engineering, feature store build
3	Modeling & calibration (Weeks 7–9)	XGBoost training, Platt scaling, lead-lag analysis, initial signal generation
4	Dashboard & evaluation (Weeks 10–12)	Streamlit dashboard, backtesting module, precision/recall eval, final report and presentation

4 Evaluation

The system will be evaluated across three dimensions. First, extraction quality: a manually labeled subset of transcripts will be used to assess the accuracy of the Claude API’s structured sentiment and theme extraction against human annotation. Second, retrieval quality: precision and recall on a labeled subset of features, assessing the system’s ability to surface contract-relevant content from earnings transcripts. Third, signal quality: directional accuracy of the mispricing signal against realized earnings outcomes and subsequent contract price movements in backtesting.

5 Technology Stack

Layer	Tools
Ingestion	SEC EDGAR REST API, FRED API, Kalshi API, Polymarket API
Orchestration	Apache Airflow (scheduled DAG, nightly cadence)
Storage	Amazon S3 (raw data), PostgreSQL (feature store)
Transformation	dbt (staging to feature store transforms, schema tests)
NLP & Features	Claude API (structured LLM extraction), spaCy (preprocessing)
Modeling	XGBoost, LightGBM, scikit-learn (Platt scaling)
Output	Streamlit (dashboard), matplotlib (backtesting charts)
Version control	Git / GitHub (public repository with README)

6 Portfolio Notes

This project is designed to demonstrate end-to-end data engineering and data science competency across a single production-grade codebase. Key differentiators relative to typical academic projects:

- **Live, scheduled pipeline** — not a static notebook. The system runs continuously via Airflow and reflects current data at any point in time.
- **Mispricing framing** — model output is contextualized against a live market price, demonstrating quantitative finance intuition beyond standard classification tasks.
- **LLM integration is justified** — the Claude API is used specifically for structured extraction from earnings transcripts, a task where traditional NLP underperforms. This is not LLM for the sake of LLM.
- **dbt-managed transformations** — raw-to-feature-store transforms are version-controlled and tested, reflecting real-world data engineering workflows.
- **Probability calibration** — Platt scaling ensures model output is interpretable as true probabilities, a statistical rigor uncommon in student projects.
- **Public GitHub repository** with clean documentation will be maintained throughout development.